

ARISS Clock - Helper

By: Ken McCaughey (N3FZX)

On: 2024-12-12

Ver 2.0.0

This file provides helpful information for users new to Python to help run the ARISS Clock. You do not need to know Python to run this!

The README file has information on how the tool works. This file has information on how to setup Python and run the tool. This is where known issues are documented and additional tips reside.

Contents

- Python
 - Installing Python Libraries
 - Running `ARISS_Clock.py`
- Thonny Python IDE
 - Installing Thonny on a Mac
 - Thonny Virtual Environment Setup
 - How To Add Python Libraries in Thonny
 - Setup `ARISS_Clock.py` in Thonny
 - Running `ARISS_Clock.py` in Thonny
- Creating a Native Executable
 - Install `pyinstaller`
 - Running `pyinstaller`
- Known Issues and Tips

Python

Python comes with Linux and Raspberry Pi's. Macs often have Python, but it is not configured for user projects. Windows doesn't come with Python.

Linux and Raspberry Pi systems also come with Libre Office needed to open MS-Word `.docx` files. Thonny is in the Raspberry Pi software repository and most Linux repositories. These systems are generally easier to setup to use this tool. Libre Office is available for Windows and Macs in addition to Linux.

The two sections below assume Python is already installed on your system. If you are a beginner or Python is not installed, skip the the next section and consider installing Thonny.

Installing Python Libraries

This tool requires a number Python library that may not normally be included with a Python installation.

```
sys
os
platform
getopt
tkinter
time
datetime
re
```

Only install if Python complains of missing libraries. Then only install what's missing.

These might need to be added. How they get added depended on your setup. In general they can be installed using the command line command: `pip install <name>` . If you are using a virtual environment (i.e. `venv`), execute the command in that folder with the virtual environment active.

Alternatively, use Thonny and its tool to install library packages. This is detailed below.

Running `ARISS_Clock.py`

To run the tool, open a terminal and at the command line enter:

```
python3 ARISS_Clock.py
```

Thonny Python IDE

Thonny is a decent basic Python Integrated Development Environment (IDE). It is free and runs on Linux (and Raspberry Pi's), Macs, and Windows. It also bring along its own Python installation. Installing Thonny gets you a good tool and Python in one step. This is recommended for Python beginners. The software may be in your machine's software repository (or store). Or the files for your OS can be found at <https://thonny.org/> . The wiki for Thonny can be found at <https://github.com/thonny/thonny/wiki> .

For Windows and the Mac it is best to install for current user only, not all users. This should not require admin privileges.

Once installed, turn on the file viewer. In Thonny, from the main toolbar click on **View** then click to add a check mark for **Files** . It will add a sub-window to the left side. When you are running the script this should be set to your working directory with all the ARISS Moderator Script files.

Installing Thonny on a Mac

A good guide for installing Thonny on Mac is at the link below. It also has some instructions for adding libraries.

<https://www2.seas.gwu.edu/~cs4all/1012/editor-install/thonny-mac.html>

Thonny Virtual Environment Setup

Starting with Python version 3.11, a virtual environment is required. This is in essence a local container of the Python files for users to use. It isolates any Python files the operating system may be using to protect your OS. Thonny supports the virtual environment. This needs to be setup only once for Thonny. It can use used for all your Python projects.

Note that the virtual environment setup is not required under Windows.

Start by making a Python project folder, i.e. **Python_Projects** . Within the folder create a new empty folder called **venv** . This will be the location for the user virtual environment, which will be setup below.

In Thonny, from the main toolbar click on **Tools** then **Options** . It will open a window. Select the **Interpreter** tab.

Which kind of interpreter... should be **Local Python 3** . If not click on the drop down menu and select **Local Python 3** .

At the bottom right of the window find `New virtual environment`. Click on that, and select the `venv` folder created above.

Now set Thonny to use that virtual environment. `Python executable` has a drop down menu. Click on it and find the path that corresponds to the `venv` folder. Note that the path will end with something like `.../Python_projects/venv/bin/python3`. Click on `OK` to close the `Thonny Options` window.

How To Add Python Libraries in Thonny

In Thonny, from the main toolbar click on `Tools` then `Manage packages...`. A window will open up. All the installed packages are listed in a column on the left. These are all packages only installed in the virtual environment.

In the box at the top you can enter the name of the package (or library) needed. It will provide a list of matches under `Search results`. Click on the one you need. It will then give you a more detailed description with an option to `Install`. Click on `Install`. A small window will appear as it is installed. Once complete it will appear on the list column on the left. There is also an option to `Uninstall`. Note that if it was already installed, there may be an options to `Upgrade` if a newer version has been released. Click `Close` when done.

Setup `ARISS_Clock.py` in Thonny

Download the Zipped package from GitHub at:

```
https://github.com/twk6809/ARISS-Clock
```

Unzip the GitHub file `ARISS_Clock-main.zip`. Find the the `ARISS_Clock_V2.0.0` folder and copy to the `Python_projects` folder.

Running `ARISS_clock.py` in Thonny

It is possible to associate Python files (with the `.py` ext) with Thonny. The method varies with OS, so it is not included here. If you do this, a double click on any `.py` file can open it up in Thonny automatically.

Open Thonny and on the main toolbar click on `File`, then `Open` and work your way to the folder with the file `ARISS_Clock.py` and open the file. It will open the Python script in its own tab.

In the Thonny Files sub-window (left side) you should see all the ARISS Moderator Script files.

Any of the sub-windows in Thonny can be resized. Just grab the edges with the mouse and drag to suit.

Thonny can open and edit text files, such as the `ARISS_Clock_config.txt` file. It can open the `ARISS_Clock_readme.txt` file as well.

To run the script, just click on the `Run` button (green circle with right arrow) on the toolbar. Or on the toolbar click on the `Run` menu, then click on `Run current script...`.

To run with any of the command line switches (see the README) then it needs to be run from the shell. In the Thonny `Shell` sub-window at the prompt:

```
>>> %Run ARISS_Clock.py <options>
```

For example to have the Clocks above the timers enter:

```
>>> %Run ARISS_Clock.py -T
```

For the list of options enter:

```
>>> %Run ARISS_Clock.py -h
```

Then just minimize Thonny. If you close Thonny you close the `ARISS_Clock` too.

Creating a Native Executable

A native binary may be included on the GitHub page for this project. If it exists, it will be in a separate folder that denotes the OS. If it exists, download it and place the binary file in the `ARISS_Clock` folder with all the other files. If it is not already available for your OS, you can create one with the steps below.

It is possible to make an executable for your operating system. You need to have Python installed. It can be done with Thonny. The advantage is that you can just run it without having to fuss this Python or Thonny. There is some limited portability to copy and run the executable on other machines. The disadvantage is that you need to create and test a new executable with every update that comes out.

Creating a native executable requires an extra file, `ARISS_logo_simple.ico`.

Install `pyinstaller`

To install the `pyinstaller` library needed by the ARISS script generator, in Thonny, from the main toolbar click on `Tools` then `Manage packages...`. In the search box, enter `pyinstaller`. Click on `pyinstaller` in the search results. Click on `Install`. Click `Close` when done.

Running `pyinstaller`

Below are the command line instructions for making a native binary executable for most operating systems. Below are the commands to be run at the command line. A bunch of messages will scroll by and, if successful, end with something similar to the following:

```
##### INFO: Building EXE from EXE-00.toc completed successfully.
```

This will also create a file called `ARISS_Clock.spec`. The file has the parameters from the last time the `pyinstaller` was run.

In all cases additional two folders will be created, `build` and `dist`. The files in the `build` do not need to be saved. The file in the `dist` folder is the native binary executable. This file should be copied to the with all the other ARISS Clock files. A shortcut can be made if desired, but setup to run in a terminal window.

To run the native executable, in a terminal window, enter `ARISS_Clock`.

Make a Linux (or Raspberry Pi) Executable

- In Thonny on the main toolbar click on `Tools`, then `Open System Shell...` This should open the terminal window with a command line in the folder with the ARISS Clock files.
- On Linux/Rasp Pi command line (all on one line):

```
pyinstaller --onefile -w -F -i "ARISS_logo_simple.ico" --add-data  
'ARISS_logo.png:.' ARISS_Clock.py
```

Make a Windows Executable

- In Thonny on the main toolbar click on `Tools`, then `Open System Shell...` This should open the terminal window with a command line in the folder with the ARISS Clock files.
- On Windows command line (all on one line):

```
pyinstaller -w -F -i "ARISS_logo_simple.ico" --add-data ARISS_logo.png;. --add-  
data ARISS_logo_simple.ico;. ARISS_Clock.py
```

Make a Mac Executable

- In Thonny on the main toolbar click on `Tools`, then `Open System Shell...` This should open the terminal window with a command line in the folder with the ARISS Clock files.
- On Mac command line (all on one line):

```
pyinstaller -w -F -i "ARISS_logo_simple.ico" --add-data ARISS_logo.png;. --add-  
data ARISS_logo_simple.ico;. ARISS_Clock.py
```

Known Issues and Tips

If you get errors or have issues take a look below first.

There might be an issue with clocks or timers being off by an hour due to the settings for daylight savings time on the computer. A fix might require changing your daylight savings setting. Times and timers are known to be correct for standard time. Please report any issues with times or timers that appear to be off by one hour.

Be aware the native executables for Windows and the Mac might trip virus protection software.

If running in Thonny, make sure the `ARISS_Clock` tab is active before clicking on `Run`. If another tab, such as the config file is open and the active tab, it will try to run what is in the form, which is not Python, and generate lots of errors. This is easy to do! Just make the tab with the script the active one and run it.

Misspelled field or variable name(s) in the form, or the template, will not get properly populated. Do not change the field or variable names. If new ones are needed, contact the author.

If the config file becomes corrupted the tool will fail. Corruptions could be an inadvertently changed variable name, invalid date/time format, a carriage return (line break) in the middle of a variable. One sure way to recover is to delete the form file and make a new one from the `ARISS_Clock_config.txt` file.